

# Static website generator

*Calepin* can turn a directory of Typst documents into a website. In fact, the website you are reading now was entirely generated with *Calepin*.

## New website

Use this command to scaffold an example site:

```
calepin new website my_site/ --theme calepin
```

This creates a website source directory with enough structure to exercise the bundled themes:

- `my_site/calepin.toml`: the site configuration file, where you set the title, base URL, navigation, theme, and output options.
- `my_site/index.typ`: the home page source file, which builds to `index.html`.
- `my_site/404.typ`: the not-found page source file, used by static hosts such as GitHub Pages for missing routes.
- `my_site/about.typ`, `my_site/guide/*.typ`, and `my_site/fr/*.typ`: regular pages in two languages, with site menu and sidebar entries.
- `my_site/blog.typ` and `my_site/posts/*.typ`: a small blog index and post source files using `calepin.pages()`.

The `--theme` value selects the scaffold to create. Use `--theme academic` for the academic starter. The scaffold includes footnotes, code output, tables, top navigation, sidebar navigation, language metadata, posts, and PDF twins for every page.

## Build

Compile a website directory with the same compile command used for a single Typst document:

```
calepin compile my_site/
```

This compiles the website in place: source files are read from `my_site/`, and generated outputs are written back into `my_site/`.

To compile the same source directory somewhere else, pass an output directory as the second path:

```
calepin compile my_site/ public/
```

When the input path is a directory, *Calepin* looks for `calepin.toml` at the root of that directory. An explicit `--config <path>` supersedes the automatic lookup; if no config is found either way, the build fails.

By default, website pages render to HTML. Configure PDF output and site settings in Site configuration.

## Serve

Serve the built site locally with:

```
calepin serve my_site/
```

This is useful for a quick preview after `calepin compile`. By default, *Calepin* uses `127.0.0.1` and the first available port from 8000.

Use `--host` and `--port` when you need a specific address:

```
calepin serve my_site/ --host 127.0.0.1 --port 8001
```

Add `--open` to launch the served site in your default browser:

```
calepin serve my_site/ --open
```

## Watch

During development, watch the website source directory for changes, and re-compile automatically:

```
calepin watch my_site/ my_site/
```

As with `compile`, the first positional argument is the source directory and the optional second positional argument is the output directory. The first build renders the whole site. After that, *Calepin* hashes each edited page and rebuilds only the pages whose content actually changed; files that were touched but not modified are skipped.

Changes that can affect more than one page trigger a full rebuild instead. This includes edits to `calepin.toml`, themes, or assets, pages that are added, removed, or renamed, and edits that change the site navigation or page metadata.

Add `--serve` to run the local server while watching; it uses the same `--host`, `--port`, and `--open` options as `calepin serve`.

```
calepin watch my_site/ my_site/ --serve --open
```

## Features

- Directory builds from Typst source files.
- In-place or separate output directories.
- Sidebar and site menu navigation from configuration or discovered pages.
- HTML output with optional PDF twins.
- Page metadata and `calepin.pages()` for listings and indexes.
- Multilingual site navigation.
- Static file copying for assets, downloads, and host-specific files.
- Generated `sitemap.xml` and `robots.txt`, with template overrides for `robots.txt`.
- Optional Pagefind search index generation, with cached no-op rebuilds.
- Optional HTML minification, including inline CSS and JavaScript.
- Local serving and incremental watch builds.